
IMPLEMENTAÇÃO DE CONTROLE REMOTO WEB DE UM ROBÔ INTEGRADO AO ROS

Arthur Lucas Costa Ciriaco da Silva e Caio Miguel Leite de Oliveira - 3º ano do Ensino Médio

Leonardo Rodrigues de Lima Teixeira

leonardo.teixeira@ufrn.br

Escola Agrícola de Jundiaí/ Universidade Federal do Rio Grande do Norte

Resumo: É inegável que, atualmente, a robótica se destaca como uma das principais formas de inserção dos jovens no universo da tecnologia. Através de projetos envolvendo a robótica, os jovens têm a oportunidade de aprender e se aprofundar em conceitos de mecânica, eletrônica, física e diversas outras áreas correlatas. Este trabalho tem como objetivo realizar uma prova de conceito de um sistema de controle remoto via interface web de um robô integrado ao ROS (Robot Operating System), permitindo a definição da direção de um servomotor frontal e ainda monitorar a distância adquirida por um sensor ultrassônico.

Palavras Chaves: Tecnologia, Sistema, Robótica, ROS.

Abstract: *It is undeniable that, currently, robotics stands out as one of the main ways for young people to enter the world of technology. Through projects involving robotics, young people have the opportunity to learn and delve deeper into concepts of mechanics, electronics, physics and several other related areas. This work aims to carry out a proof of concept of a remote control system via the web interface of a robot integrated with ROS (Robot Operating System), allowing the definition of the direction of a frontal servomotor and also monitoring the distance acquired by an ultrasonic sensor.*

Keywords: *Technology, System, Robotics, ROS.*

1 INTRODUÇÃO

Robótica é o estudo dos robôs, o que significa que é o estudo da sua capacidade de sentir e agir no mundo físico de forma autônoma e intencional. É um campo em crescimento, cuja definição foi evoluindo ao longo do tempo, e envolve ter autonomia, sentir, agir e atingir metas no mundo físico (MATARIC, 2014).

O ROS é uma estrutura (*framework*) flexível de código aberto para desenvolver softwares para robôs. É uma coleção de ferramentas, bibliotecas e convenções que visam facilitar tarefas de criação e desenvolvimento de robôs complexos e robustos para serem utilizados nas mais diversas plataformas robóticas. Ele fornece os serviços normalmente utilizados em um sistema operacional, como abstração de hardware, controle de dispositivo de baixo nível, implementação de funcionalidade comumente usada, passagem de mensagens entre processos e gerenciamento de pacotes (QUIGLEY *et al.*, 2009).

O principal recurso do ROS é a forma como o software é executado e a maneira como ele se comunica, permitindo que se projete software complexo sem saber como determinado hardware funciona. O ROS fornece uma maneira de conectar uma rede de processos (nós, em inglês *nodes*) a um hub central. Os nós podem ser executados em vários dispositivos e se conectam a esse hub na topologia de ponto a ponto (QUIGLEY *et al.*, 2009).

Em suma, ROS possui um conjunto de ferramentas de software, bibliotecas e convenções, com o objetivo de simplificar a tarefa de desenvolver softwares para um robô. Isso facilita e agiliza a implementação, não sendo necessário recomençar a cada novo trabalho (TSARDOULIAS; MITKAS, 2017).

Nesse artigo será trabalhado o processo por trás da implementação de um sistema de controle remoto via interface web integrada ao ROS, o qual serve de validação do ROS, com o intuito de trazer esse sistema para o nosso ambiente institucional para contribuir com futuras pesquisas relacionadas à essa implementação no meio da robótica principalmente, mas tendo a serventia de aplicações no meio industrial, agrícolas, com a automatização e controle de robôs por exemplo.

2 O TRABALHO PROPOSTO

Este trabalho foi desenvolvido com o objetivo de analisar como funciona a comunicação/implementação do Ros ao robô. Com isso, foi possível desenvolver uma interface capaz de fazer o controle dos movimentos do robô, visualizar dados sensoriais em tempo real(sensor ultrassônico).

Foi proposto realizar uma validação de integração do robô com o ROS, a partir de um sistema que envia comandos da interface web para o robô por meio de um servidor intermediário, que traduz as interações para mensagens compatíveis com o ROS. O ROS, por sua vez, gerencia os sensores e atuadores do robô, executando as ações solicitadas e retornando informações para a interface.

3 MATERIAIS E MÉTODOS

Durante todo o desenvolvimento do projeto, antes de realizar qualquer ação que fosse custosa, era necessário um estudo aprofundado e a utilização de simuladores e modeladores 3D, a fim de entender qual seria o resultado esperado pelo modelo antes mesmo de montá-lo. Após alguns rascunhos realizados à mão (Figura 1), partiu-se para a

criação de um modelo 3D para o robô idealizado (Figura 2). Para tal, foi utilizado o software Tinkercad.

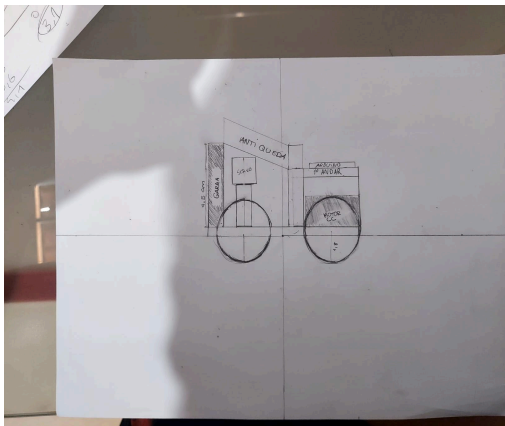


Figura 1 – Desenho do modelo feito à mão.

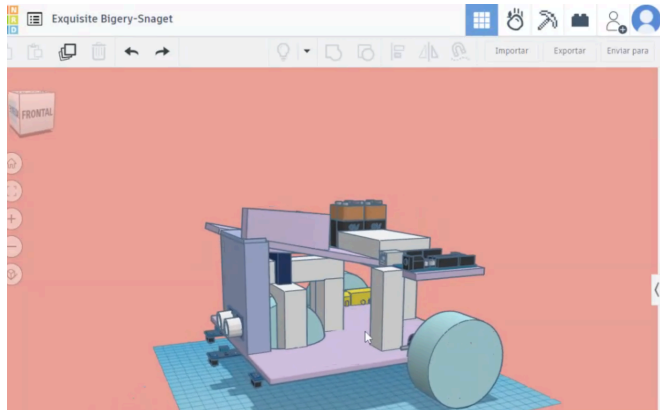


Figura 2 – Modelagem 3D do robô.

O robô foi impresso em material PLA através de uma impressora 3D Creality CR2-V2.

Com o objetivo de realizar o projeto, adicionou-se ao robô uma placa microcontroladora Arduino Mega, modelo ATmega2560-16AU, com 3 sensores de cor modelo Tcs230, 4 pilhas de 5 Volts cada, 1 servomotor frontal, 2 motores de corrente contínua na parte traseira, 1 sensor ultrassônico HC-SR04, 1 protoboard e por fim, um módulo ponte H modelo L298N.

Se tratando da implementação do ROS, no laboratório foi disponibilizada uma Shield Ethernet, a qual foi escolhida, pois a Unidade Acadêmica estava sofrendo problemas de Internet sem fio (Wi-Fi), impossibilitando o uso do ESP32 por exemplo, com o intuito de estabelecer a comunicação entre o computador instalado com o ROS e o robô.

Dando continuidade ao ROS, foi necessário levar em consideração que existem dois tipos de ROS. O ROS1(Rosserial) foi designado para projetos que envolvam a robótica. Enquanto o ROS2 é uma estrutura mais “complexa” que tem o objetivo de atender as necessidades modernas, como exemplo a comunicação em tempo real, confiabilidade e maior flexibilidade. Com isso, foi criado uma tabela indicando suas principais diferenças(Figura 3).

ASPECTO	ROS1	ROS2
Arquitetura	Centralizada (rosmaster)	Descentralizada (DDS)
Tempo Real	Não suportado	Suportado
Segurança	Limitada	Integrada
Suporte Plataformas	Principalmente Linux	Linux, Windows, macOS
QoS	Não disponível	Suportado
Estado da Comunidade	Focado no legado	Foco principalmente no futuro

Figura 3- Tabela mostrando as principais diferenças entre o ROS1 e ROS2.

Então por que utilizar o ROS1, ao invés do ROS2? já que o ROS2 teoricamente é melhor?

O uso do rosserial em vez do ROS 2 com um Arduino Mega ocorre devido a várias diferenças fundamentais entre os dois sistemas e suas aplicações. O Arduino Mega possui recursos limitados, como 8KB de RAM e um microcontrolador AVR de 16 MHz, que são suficientes para o rosserial, mas insuficientes para atender aos requisitos mínimos do ROS 2, que foi projetado para sistemas mais modernos e robustos. O rosserial foi desenvolvido especificamente para suportar dispositivos embarcados simples, como o Arduino Mega, utilizando comunicação serial eficiente com o host que executa o ROS.

Além disso, o rosserial utiliza uma interface serial simples, que pode ser feita via USB ou UART, ideal para dispositivos com capacidades computacionais limitadas. Em contraste, o ROS 2 usa o protocolo DDS (Data Distribution Service), que exige maior capacidade de processamento e memória, tornando-o incompatível com o hardware do Arduino Mega sem modificações ou hardware intermediário.

Então, quando considerar o uso do ROS 2?

Se você precisar de um sistema distribuído, com maior confiabilidade e comunicação robusta (por exemplo, com QoS garantido), pode ser necessário migrar para o ROS 2. Nesse caso, usar um hardware mais moderno, como ESP32 ou STM32, seria recomendável.

As etapas de instalação foram seguidas pelo site oficial do próprio ROS e foram feitas como descrito a seguir.

Para instalar o ROS e configurar o Arduino, primeiro é necessário configurar o ambiente ROS no computador. Deve-se escolher uma versão do ROS compatível com o sistema operacional, portanto, foi utilizado o Ubuntu 20.04 Focal Fossa, e seguir o guia oficial de instalação.

Em seguida, é preciso instalar o pacote roserial, que permite a comunicação entre o ROS e microcontroladores como o Arduino. Abre-se um terminal e instala-se o pacote roserial com o comando apropriado, substituindo '<sua_distribuição>' pela versão do ROS que está sendo usada. Depois, gera-se a biblioteca ros_lib executando um comando no terminal, o que cria uma pasta chamada ros_lib que deve ser copiada para o diretório libraries da IDE do Arduino.

Agora, pode-se programar o Arduino para se comunicar com o ROS. Um exemplo básico de código para o Arduino inclui a inclusão das bibliotecas necessárias, a inicialização do nó ROS e a publicação de uma mensagem. Finalmente, no computador, inicia-se o roscore e o nó roserial, certificando-se de substituir a porta correta onde o Arduino está conectado. Seguindo esses passos, consegue-se configurar o ROS para trabalhar com o Arduino usando a biblioteca roserial. Segue o fluxograma indicando os passos para a sua instalação(Figura 4)

Para a comunicação robô-ROS, foi criada uma página html com botões para fazer o controle do servomotor. Na Figura 5, pode-se observar a página html e o ROS TOPIC listando os últimos tópicos publicados no servidor.

O servomotor foi muito importante para o projeto. Com isso, foram necessárias pesquisas para entender o seu funcionamento integrado ao Arduino e como seria feito o seu controle.

Foi utilizada a definição de TANNUS (2018, p1) “Servomotores são dispositivos compostos por uma combinação de quatro componentes: um motor de corrente contínua, engrenagens, circuito de controle e um potenciômetro”. Em outras palavras, trata-se de um dispositivo eletromecânico projetado para realizar movimentos com precisão, permitindo o giro em ângulos ou distâncias específicas, garantindo o posicionamento exato. O servomotor SG90 é composto por engrenagens de redução em nylon e tem um ângulo de operação que varia de 0° a 180°.

Vale ressaltar a utilização do sensor ultrassônico, que tinha como objetivo demonstrar o seu funcionamento integrado ao atuador de maneira simultânea. O HC-SR04 é um sensor ultrassônico de distância, composto por um emissor e um receptor, capaz de medir distâncias que variam de aproximadamente 2 cm a 4 m, com uma precisão de cerca de 3 mm. O sensor emite sinais ultrassônicos que se refletem no objeto alvo e retornam ao sensor, permitindo calcular a distância com base no tempo de trânsito do sinal. É amplamente utilizado como detector de objetos, especialmente na robótica, onde serve para identificar obstáculos e ajustar continuamente o percurso de um robô.

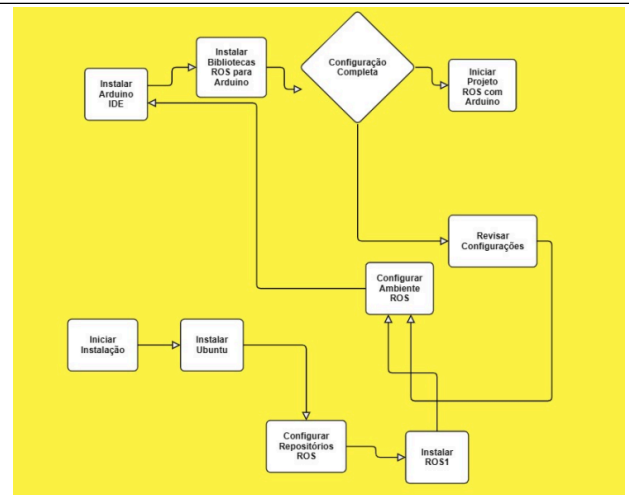


Figura 4- Etapas de instalação do roserial.

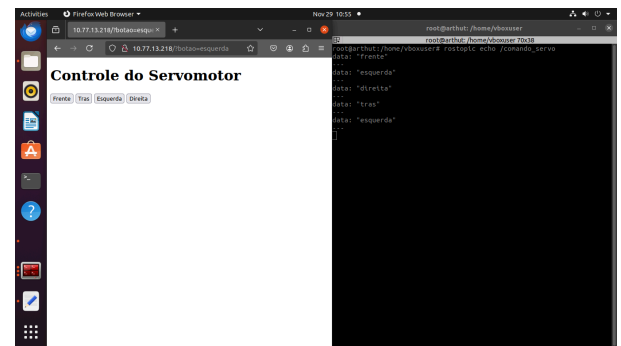


Figura 5- Página html para o controle do servomotor.

4 RESULTADOS E DISCUSSÃO

Os resultados obtidos nesta pesquisa são apresentados em dois momentos distintos. No primeiro, realiza-se uma análise detalhada do projeto, avaliando se as metas estabelecidas e os objetivos propostos foram alcançados de maneira eficaz. No segundo momento, busca-se compreender se o trabalho realizado trouxe uma contribuição significativa para a instituição, além de verificar o impacto potencial em estudos e pesquisas futuras. Essa abordagem permite não apenas mensurar o sucesso do projeto em termos práticos, mas também refletir sobre o legado acadêmico e científico deixado para o avanço do conhecimento na área.

Diante do resultado, pode se dizer que as metas e os resultados foram alcançados, pois o robô-ROS foi validado(Figura 6).

No início do projeto, enfrentamos dificuldades na implementação no robô devido à falta de pesquisas suficientes que fornecessem um embasamento teórico adequado. Para superar esse desafio, ao longo do desenvolvimento, foi recorrido a diversas fontes, como vídeos e artigos em outros idiomas, o que permitiu a continuidade da validação. O primeiro passo do processo foi realizar a validação do servomotor, etapa essencial para o avanço do projeto.

Dando continuidade ao projeto, foram feitos testes antes de implementar ao robô. No caso, o atuador e o sensor foram testados primeiro sem o robô, os resultados foram satisfatórios e no final das contas foi implementado ao robô (Figura 7).

Para explicar melhor a funcionalidade do robô-ROS, foi feito um fluxograma, indicando as etapas de seu funcionamento(Figura 8).

As contribuições do robô baseado no ROS (Robot Operating System) são amplas e impactantes, abrangendo diversos setores e promovendo avanços significativos na robótica moderna. Este projeto vai além do ambiente institucional, beneficiando desenvolvedores e entusiastas da robótica que estão constantemente em busca de inovações.

No âmbito industrial, a aplicação de sistemas robóticos controlados por ROS pode ser observada em linhas de montagem automatizadas, onde a precisão e a eficiência são cruciais. Na agricultura, a automação e o monitoramento remoto de atividades, como o plantio e a irrigação, tornam-se mais acessíveis e otimizados graças a essas tecnologias.

No campo científico, educacional e tecnológico, a contribuição é ainda mais notável. Este tipo de sistema é ideal para uso em pesquisas e em ambientes educacionais, pois proporciona uma oportunidade única de aprendizado prático e interdisciplinar. Ele incentiva o estudo aprofundado de tecnologias como o ROS, interfaces web, e o controle de servomotores, que são pilares fundamentais da robótica moderna. Além disso, o projeto preenche uma lacuna existente na literatura acadêmica, já que, como mencionado, ainda não há uma quantidade significativa de pesquisas aprofundadas nessa área. Com isso, ele não apenas fomenta o conhecimento, mas também promove a criação de novos projetos e inovações.

Outra contribuição significativa é o impacto na democratização do acesso à robótica. A integração com o ROS e interfaces web permite a construção de soluções mais acessíveis e adaptáveis, favorecendo tanto pequenas iniciativas quanto grandes indústrias. Além disso, ao adotar padrões abertos, o projeto fortalece a colaboração dentro da comunidade global de desenvolvedores, incentivando a troca de ideias e o desenvolvimento de tecnologias cada vez mais avançadas.

Portanto, o robô-ROS não apenas avança o estado da arte na robótica, mas também contribui de maneira estratégica para a inovação, o aprendizado e a colaboração em múltiplos campos de aplicação.

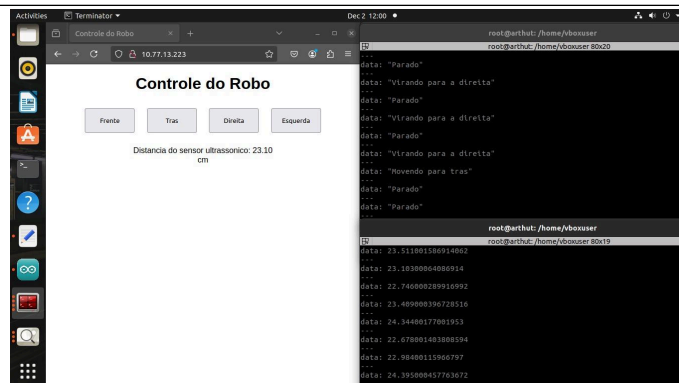


Figura 6- Controle Validado do Robô-ROS.

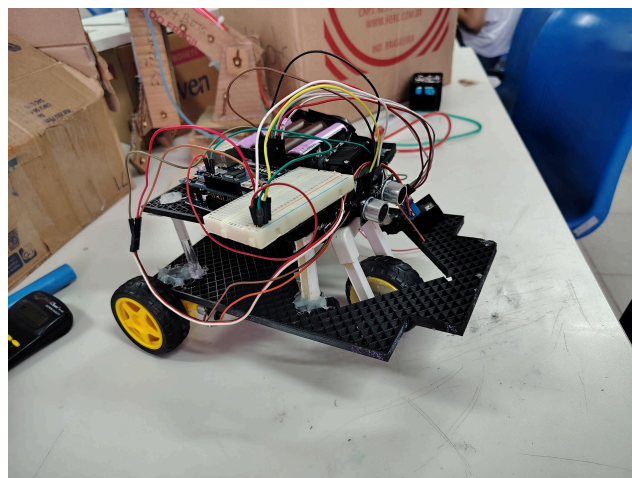


Figura 7- Robô-ROS pronto e validado.

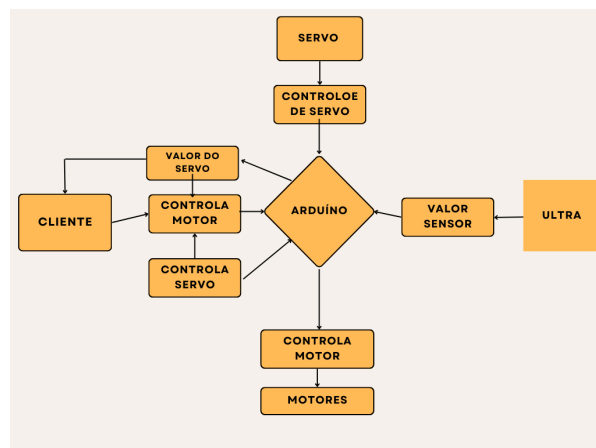


Figura 8- Fluxograma mostrando as etapas de seu funcionamento.

5 CONCLUSÃO

Para a realização do trabalho, foi necessária a utilização de diversas ferramentas e técnicas para a sua execução. Além disso, foi de suma importância a divisão do trabalho entre os membros da equipe, haja vista que os mesmos se separaram de

acordo com as suas especialidades, se dividindo em três formas: Pesquisas a respeito do ROS, programação e circuito do robô-ROS.

Pode-se dizer que um dos maiores pontos positivos é o conhecimento que essa pesquisa pode criar tanto para os estudantes, quanto para aqueles que lêem o artigo, uma vez que os métodos utilizados durante a pesquisa, podem ser aproveitados para que outras pessoas interessadas no assunto possam utilizar como base.

Em primeiro plano, a gestão do tempo foi muito bem distribuída ao longo do trabalho, tendo em vista que o trabalho foi realizado com um prazo “apertado”, entretanto com o pouco tempo que se teve, foi importante a organização de tempo e a divisão de tarefas e isso fluiu muito bem.

A implementação do ROS foi um grande desafio, especialmente considerando que os alunos precisavam aprender seu funcionamento ao longo do projeto. O principal obstáculo enfrentado foi a escassez de pesquisas específicas sobre a integração de robôs com o ROS, o que tornou o processo ainda mais complexo.

Afinal de contas, o controle remoto poderia ser feito sem o ROS?

Depois de uma série de pesquisas, foi constatado que poderia ser feito sem a implementação do ROS, porém poderiam ocorrer atrasos na comunicação do arduino com os sensor ultrassônico e o atuador, enquanto com a aplicação do rosserial é feita de maneira simultânea.

Nesse contexto, foi indispensável o uso do ROS devido a necessidade de ter um sistema embarcado hard real-time, portanto era necessário prazos rígidos que fizessem a leitura e envio de informações ao mesmo tempo em uma execução paralela. Sem o rosserial, isso poderia ser feito através de uma simulação da leitura paralela, porém, perderia um pouco da precisão, portanto diante dos fatores de aumentar a precisão o ROS foi introduzido pois ele consegue ler os circuitos e mandar sinais elétricos para os atuadores devido a sua estrutura de nós.

Portanto, este artigo se torna especialmente relevante, pois busca preencher essa lacuna e contribuir para futuros pesquisadores e desenvolvedores interessados em explorar essa área.

REFERÊNCIAS BIBLIOGRÁFICAS

QUIGLEY, M.; GERKEY, B.; CONLEY, K.; FAUST, J.; FOOTE, T.; LEIBS, J.; BERGER, E.; WHEELER, R.; NG, A. ROS: an open-source Robot Operating System. In: KOBE, JAPAN. ICRA workshop on open source software. [S.l.], 2009. v. 3, n. 3.2, p. 5.

MATARIC, M. J. Introdução à Robótica. São Paulo: Editora Unesp/Blucher, 2014.

TSARDOULIAS, Emmanouil; MITKAS, Pericles. Robotic frameworks, architectures and middleware comparison. ArXiv,

2017. Disponível em: <https://arxiv.org/pdf/1711.06842.pdf>. Acesso em: Dezembro de 2024..

TANNUS, A. M. Arduino: Servomotores. Centro Universitário de Anápolis UNIEVANGÉLICA, Goiás, 2018.

<https://wiki.ros.org/Documentation>